

TERRAFORM & AZURE DEVOPS MASTER STUDY GUIDE & PROJECT JOURNAL

A Comprehensive Guide from Monolith to Dynamic Pipelines

Author: Aqib Mohammad

Topic: Infrastructure as Code (IaC) & CI/CD Pipelines on Azure

Environment: macOS Developer Workspace & Azure Cloud Portal

Date: July 2026

Chapter 1: The Monolithic Start & Local Workflow

Every enterprise cloud journey begins with a monolithic prototype to test basic connectivity and syntax. In this initial phase, we wrote a single root configuration containing all the baseline resource blocks.

1.1 Baseline Resources Deployed

- **Resource Group:** The foundational logical container for all assets.
- **Virtual Network (VNet):** Private isolated network address space.
- **Subnets:** Separated subnets for frontend and backend access control.
- **Public IP & NIC:** Network Interface card for connectivity.
- **Linux Virtual Machine:** The compute instance deployed inside the VNet.

1.2 Local Authentication & CLI Workflow

We initialized our local environment using Azure CLI. The local loop involves three core commands:

1. **terraform init:** Downloads provider plugins (like hashicorp/azurerm).
2. **terraform plan:** Generates a list of actions (add, change, destroy) by querying Azure APIs.
3. **terraform apply:** Deploys the configuration to your subscription.

1.3 Troubleshooting: The VM Hostname Underscore Error

Gotcha Encountered: In HCL, if your VM name is configured as 'frontend_vm', Azure API will reject the deployment. This is because Linux OS hostnames must comply with the DNS naming rule (RFC 1123), which allows alphanumeric characters and hyphens ('-') but strictly forbids underscores ('_'). Replacing underscores with hyphens resolved this error immediately.

1.4 Git Version Control & The Warehouse Analogy

We integrated Git to version-control the code, using a simple Warehouse Analogy:

- **Untracked Files (Unregistered Boxes):** Files that Git ignores (like local state secrets). Git cannot restore these if deleted.
- **Tracked Files (Registered Boxes):** Files added ('git add') and committed ('git commit'). Git monitors these 24/7 and records every code change.

Chapter 2: Modular Refactoring & Zero-Downtime Migration

As codebases grow, monolithic files become unmanageable. We refactored our code into clean, isolated child modules.

2.1 Module Structure & Parameter Flow

We separated our files into directories under *modules/*:

- **modules/resource-group**: Handles Resource Group creation.
- **modules/network**: Houses the VNet and Subnets.
- **modules/security**: Configures NSGs and inbound rules.
- **modules/compute**: Deploys Public IP, NIC, and VM.

To pass data between modules (like passing the Resource Group Name down to the Network module), we used variables as inputs and outputs as return values. We described this using the **Restaurant Analogy**:

***Restaurant Analogy:** The root module is the Customer sitting at the table. The child module is the Kitchen where food is prepared (encapsulated). The **outputs.tf** file represents the Chef's Counter where ready dishes are placed. Without outputs, the data remains locked inside the kitchen, and downstream resources cannot reference it.*

2.2 Zero-Downtime State Migrations (Moved Blocks)

When you move resources from a monolith to a module, their state address changes (e.g., from *azurerm_subnet.frontend* to *module.network.azure_rm_subnet.frontend*). Run plan, and Terraform will default to destroying the old resource and creating a new one, causing major downtime.

We solved this by adding HCL **moved** blocks in our root configuration:

```
moved {
  from = azurerm_virtual_network.vnet
  to   = module.network.azure_rm_virtual_network.vnet
}
```

This mapped the state record safely with **0 resources added, 0 destroyed**.

Chapter 3: Enterprise State Management & Backend Locking

For production systems, storing the state file on a local computer is a single-point-of-failure. We migrated our state to the cloud.

- **Remote State:** Configured the *azurerm* backend block to write the state to a manual Azure Blob Container (*tfstate*).
- **State Locking:** Managed natively via Azure Blob Leasing. When a developer runs a plan/apply, Azure locks the state file exclusively to prevent concurrent modifications by other team members.

Chapter 4: CI/CD Pipeline Automation (Azure DevOps)

Instead of applying changes from a local terminal, we automated the lifecycle using Azure DevOps pipelines.

4.1 Authentication: Service Connection & Workload Identity Federation

We configured a Service Connection in Azure DevOps to access our Azure Subscription. We selected **Workload Identity Federation (automatic)**. This creates a secure, passwordless trust relationship using a federated Service Principal in Entra ID, completely eliminating the need to manage client secrets.

4.2 The Pipeline YAML Configuration (*azure-pipelines.yml*)

We wrote the pipeline code to run a temporary hosted Ubuntu VM, install the Terraform CLI, login, and execute commands automatically:

```
trigger:
- main

pool:
vmImage: 'ubuntu-latest'

steps:
- task: TerraformInstaller@1
inputs:
terraformVersion: 'latest'

- task: TerraformTaskV4@4
displayName: 'Terraform Init & Backend Sync'
inputs:
command: 'init'
backendServiceArm: 'azure-sp-connection'
backendAzureRmStorageAccountName: 'aqibtftstate2000'
backendAzureRmContainerName: 'tfstate'
backendAzureRmKey: 'dev.terraform.tfstate'

- task: TerraformTaskV4@4
displayName: 'Terraform Apply'
inputs:
command: 'apply'
environmentServiceNameAzureRM: 'azure-sp-connection'
commandOptions: '-var-file="dev.tfvars" -auto-approve'
```

Chapter 5: Advanced IaC Concepts (Locals & Dynamic Blocks)

In this final phase, we optimized our code using advanced programming features in Terraform.

5.1 Unified Resource Prefix (Terraform Locals)

We declared local variables (`resource_prefix = "lab01-dev"`) in our root module and passed them to child modules to dynamically tag and name resources like `'lab01-dev-frontend-vm'`.

5.2 The Shared Network Model (Scenario A)

To model real-world scenarios where VNets and Subnets are pre-deployed by a central Network Team, we transitioned our VNet resource to data blocks:

1. We ran **`terraform state rm module.network.azure_rm_virtual_network.vnet`** to remove the network from our state without deleting it from Azure.
2. We replaced `resource` blocks with **`data`** blocks to query the VNet dynamically from Azure.

5.3 Dynamic SSH Lock via HTTP Data Source

To lock SSH access to only the developer's laptop public IP, we queried `api.ipify.org` using a data source and injected the IP dynamically into the security rules:

```
data "http" "my_public_ip" {
  url = "https://api.ipify.org"
}

# Applied inside security rule source prefix:
source_address_prefix = "${data.http.my_public_ip.response_body}/32"
```

5.4 Code Optimization (Dynamic Blocks)

Instead of copying and pasting the `azurerm_network_security_rule` resource block 5 times for HTTP, HTTPS, SSH, and API ports, we wrote a map of rules in locals and looped over it using a single nested **`dynamic block`** inside the NSG:

```
dynamic "security_rule" {
  for_each = local.frontend_rules
  content {
    name = "${var.naming_prefix}-${security_rule.value.name}"
    priority = security_rule.value.priority
    direction = "Inbound"
    access = "Allow"
    protocol = "Tcp"
    source_port_range = ""
    destination_port_range = security_rule.value.destination_port_range
    source_address_prefix = security_rule.value.source_address_prefix
    destination_address_prefix = ""
  }
}
```